AI Driven Cloud Data Engineering

Machine Learning Applications & Cloud Resource Allocation

Student Name : Elena Kroupkin

Student ID : 270255004

Contents

Part A : Machine Learning Pipeline: Auto Price Training

1 Regression Analysis

This table provides a step-by-step guide to building a Linear Regression model in Azure ML Studio to predict car prices, designed for easy understanding and replication.

Preparation & Setup

Step	What to Do	How to Do It	Why It's Important
1.	**Create New Pipeline**	In Azure ML Studio, navigate to the **Designer** or Pipelines section and select "**Create New Pipeline**." Give it a descriptive name (e.g., "Auto Price Training").	This establishes the workspace for the ML workflow, helping the organize and manage the project.
2.	**Load Raw Automobile Price Data**	From the left-hand pane in Azure ML Studio, locate "**Sample data**" under "**Component** ". You can choose: - A sample dataset like "`Automobile price data (Raw)`" directly from Azure's default data. - **Upload your own data** from the local machine. - **Specify a web link** to an external data source. Drag the chosen dataset onto the pipeline canvas.	This is the initial, raw dataset that the machine learning model will use to learn and make predictions. Accessing it from various sources makes the process flexible.
3.	**Visualize Data**	**Right-click** the output of the "Automobile price data" module and select "**Preview data**." Inspect column distributions, data types, and check for missing values (e.g., in 'normalized-losses').	Performing an initial inspection helps understand the data's characteristics and identify specific columns that require cleaning or special handling before modeling.
4.	**Create Azure ML Compute Cluster**	Add a "**Create Azure ML Cluster**" module. Configure settings like: - Dedicated priority - Max 2 nodes - 120s idle scale down - Standard_DS11_v2 CPU size	Machine learning model training is computationally intensive. This cluster provides the necessary distributed processing power in Azure for efficient and scalable model training.

Data Cleaning & Pre-processing (Data Transformations)

Step	What to Do	How to Do It	Why It's Important
1.	**Select Columns in Dataset**	Connect the "Automobile price data" output to a "Select Columns in Dataset" module. Configure it to **exclude** the 'normalized-losses' column by explicitly selecting all other desired columns.	The 'normalized-losses' column often contains too many missing values, which can negatively impact model performance. Removing it ensures higher data quality for prediction.
2.	**Clean Missing Data**	Connect the output from "Select Columns in Dataset" to a "Clean Missing Data" module. Set the cleaning mode to "**Remove entire row**" for critical columns like 'bore', 'stroke', and 'horsepower'.	These columns are vital for accurate price prediction. For a few missing values, removing the entire row is often better than imputing (filling in) data, which can introduce inaccuracies.
3.	**Normalize Data**	Connect the cleaned data to a "Normalize Data" module. Set the **Transformation method to MinMax** and apply it to **Numeric column types**.	This is crucial for regression models. It scales all numeric features (typically 0-1) to prevent features with larger values from unfairly dominating the model's learning process.
4.	**Split Data**	Connect the normalized data to a "Split Data" module. Set the "Fraction of rows in the first output dataset" to **0.7 (70%)**. This creates two outputs: 70% for training and 30% for testing.	To accurately evaluate the model's real-world performance, it must be tested on data it has **never seen** during training. This ensures the model generalizes well, rather than just memorizing.

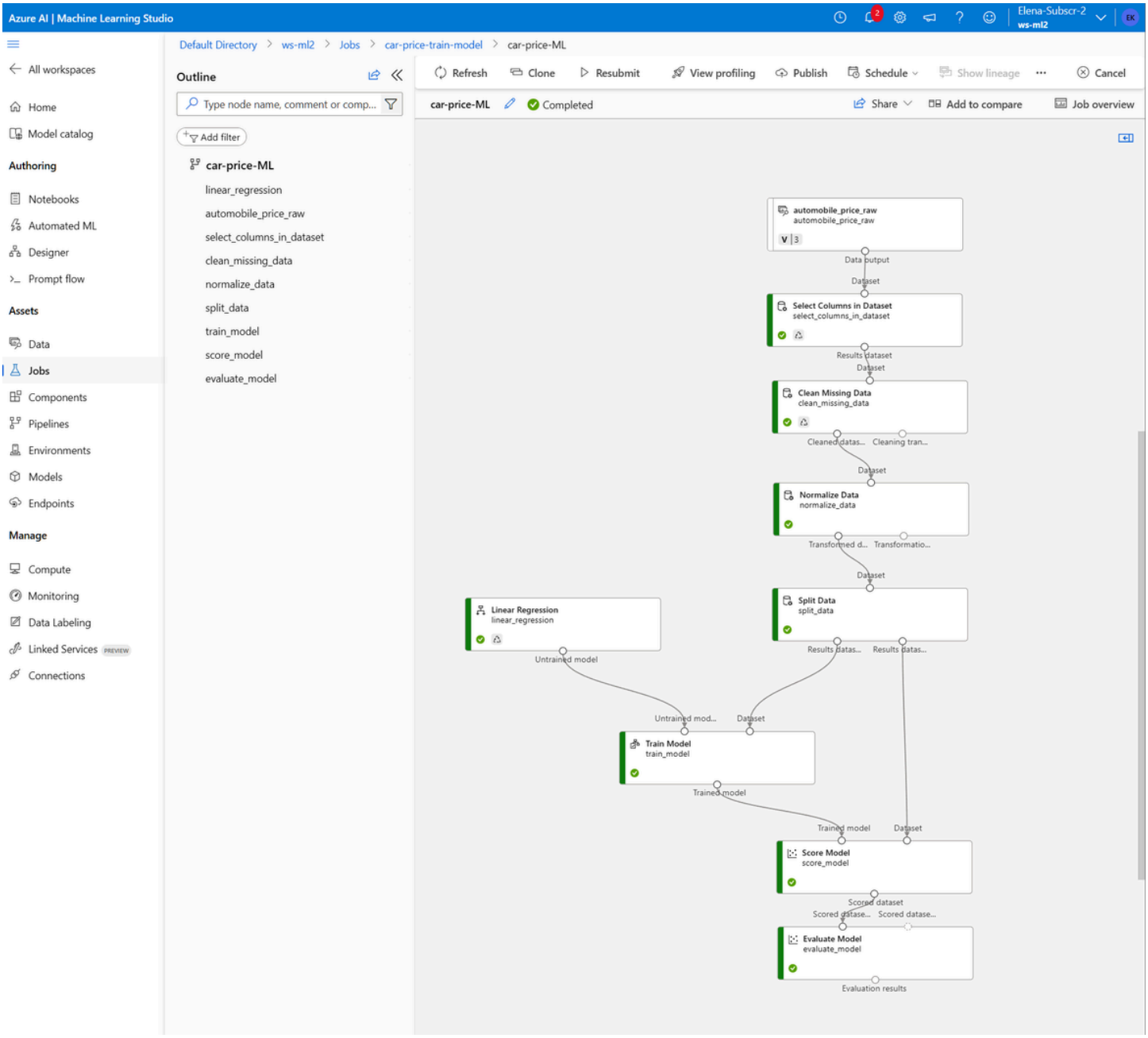
Model Training & Evaluation

Step	What to Do	How to Do It	Why It's Important
1.	**Select Linear Regression (Untrained Model)**	Drag and drop the "Linear Regression" module onto the canvas. This represents the chosen algorithm.	Linear Regression is the appropriate **supervised learning** algorithm for predicting a ***continuous numeric value*** (like car price), making it suitable for this scenario.
2.	**Train Model**	Connect the "Linear Regression" module to the **70% Training Dataset** output from the "Split Data" module. In the "Train Model" settings, specify the **Label column: price**.	This step "teaches" the model. It uses the labeled training data to learn the intricate relationships between the car's features and its actual price, forming the core of the prediction model.
3.	**Score Model**	Connect the "Trained Model" output from the "Train Model" module to the "Score Model" module. Also, connect the **30% Testing Dataset** output from "Split Data" to the "Score Model" module.	This applies the trained model to the unseen test data. It generates "Scored Labels" (predicted prices) for these new cars, simulating how the model would perform in a real-world prediction scenario.
4.	**Evaluate Model**	Connect the output from the "Score Model" module to an "Evaluate Model" module. This module will calculate performance metrics.	This step is crucial for understanding the model's accuracy. It provides key regression metrics (like MAE, RMSE, R^2) to quantitatively assess how well the model predicts prices and determine if further tuning is needed.

Azure ML Studio Implementation of the Car Price Prediction Pipeline

This screenshot displays the actual pipeline constructed within Azure Machine Learning Studio. It provides a concrete view of how the steps outlined in the table translate into a practical, deployed ML workflow in a cloud environment. It's the "blueprint" of the solution.

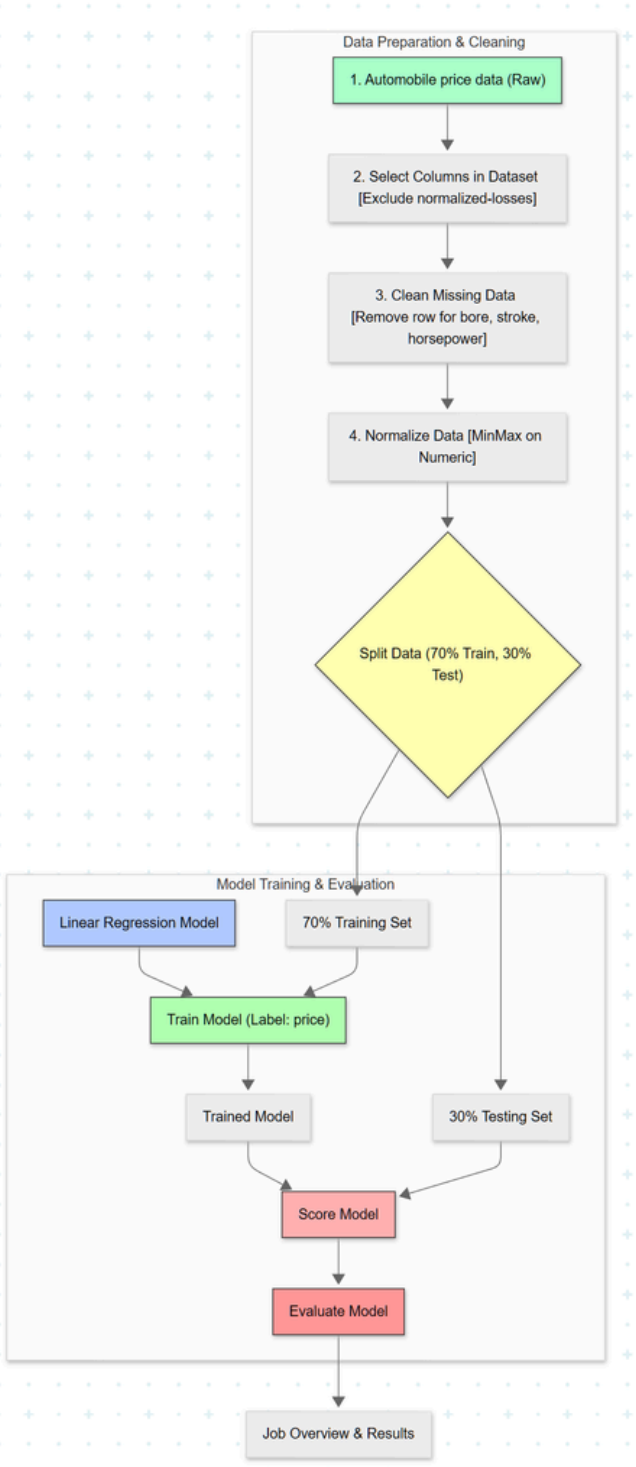
Screenshot 55: The actual pipeline



Conceptual Data Flow Diagram for the Regression Pipeline

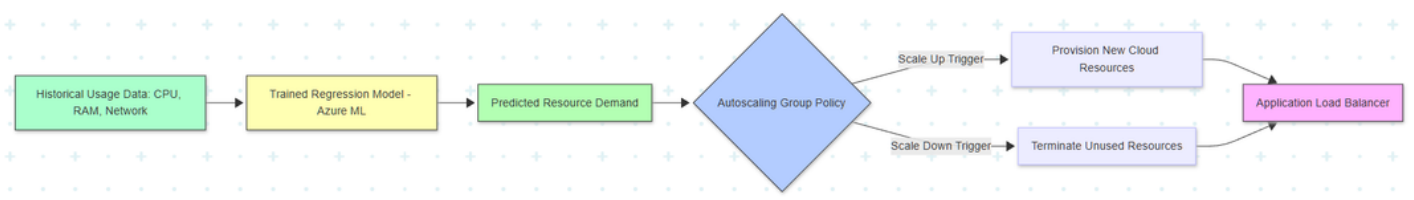
This simplified diagram illustrates the high-level stages and logical flow of data through the regression pipeline. It serves as a clear, abstract representation of the process, akin to a Data Flow Diagram (DFD). These diagrams help in understanding the overall architecture, identifying bottlenecks, and planning for efficient resource allocation.

Flow Diagram: Simplified diagram for the Regression Pipeline



This diagram visually shows how the concept of the trained regression model transitions from predicting car prices to predicting cloud resource needs, which then informs an **Autoscaling Group** (an essential component in both AWS and Azure).

Flow Diagram: ML-Driven Autoscaling .



Core Concepts & Model Justification: Linear Regression

Definition: A supervised learning algorithm predicting a continuous numeric value by finding the "line of best fit" relating features to the target.

Purpose for Scenario: Car price is a continuous value, making Linear Regression suitable for modeling how features like engine size or mileage linearly impact price.

Evaluation Metrics (Validation of Model Accuracy) : Model performance is measured on the 30% testing dataset. A better model aims for an R^2 close to 1.0 and MAE/RMSE close to 0.

Metric	Interpretation (Goal)
Mean Absolute Error (MAE)	Minimize; average dollar error in predictions. Represents the average monetary error in predictions.
Root Mean Squared Error (RMSE)	Minimize; similar to MAE but penalizes larger errors more significantly by squaring them, making it sensitive to outliers.
R-Squared (R^2)	Maximize (close to 1.0); proportion of the total variation in the target variable (price) that is explained by the model's features ("Goodness of Fit").

The process of training this regression model illustrates how ML can drive operational efficiency in a cloud environment.

Part A : Machine Learning Pipeline: Auto Price Training

2 Classification Analysis

This table provides a step-by-step guide to building a Binary Classification model in Azure ML Studio to predict loan approval, designed for easy understanding and replication.

Preparation

Step	What to Do	How to Do It	Why It's Important
1.	**Load Loan Application Data**	From the left-hand pane in Azure ML Studio, locate "**Sample data**" under "**Component** ". Choose the `loan application dataset` (e.g., from an uploaded file, a sample, or web link). Drag the chosen dataset onto the pipeline canvas.	This is the raw input dataset containing features like credit score, income, and the Loan_Status (Approved/Rejected) label, which the model will learn from.

Data Pre-processing (Data Transformations)

Step	What to Do	How to Do It	Why It's Important
1.	**Clean Missing Data**	Connect the dataset output to a "Clean Missing Data" module. Select relevant columns like 'Credit_Score', 'Income_(K)', and 'Loan_Amount'. Set the cleaning mode to "**Replace with Mean (or Median)**".	Missing values in critical features can cause errors or bias. Replacing them with the mean (or median) helps maintain data integrity while preserving as many rows as possible, suitable for these numeric columns.
2.	**Edit Metadata (Label Column)**	Connect the cleaned data to an "Edit Metadata" module. Select the Loan_Status_(Target) column. Set **Categorical: categorical** and **Fields: Labels**.	This step explicitly identifies the target variable for prediction. Marking it as a **categorical label** is crucial for classification algorithms to correctly identify the outcome. This can prevent "Train Model" errors.
3.	**Edit Metadata (Numeric Features)**	Add another "Edit Metadata" module. Select numeric features like 'Credit_Score', 'Income_(K)', and 'Loan_Amount'. Set **Data type: Float (or Double)** and **Fields: Feature**.	Ensuring numeric columns are correctly identified as features and set to a consistent data type (Float/Double) is vital for algorithms that perform mathematical operations during normalization and training.
4.	**Edit Metadata (ID Removal)**	Add another "Edit Metadata" module. Select the 'Application_ID' column. Set **Fields: Clear feature**.	The 'Application_ID' is a unique identifier, **not a predictor**. Clearing it as a feature instructs the model to ignore it during training, preventing it from incorrectly trying to learn from irrelevant IDs.
5.	**Convert to Indicator Values**	Connect the output from the last "Edit Metadata" module to a "Convert to Indicator Values" module. Select the categorical feature 'Employment_Status'. Keep **Indicator value formatting: Default**.	Machine learning models primarily work with numbers. This step transforms text-based categorical data (like "Full-Time," "Part-Time") into numerical (0 or 1) indicator columns, making it readable and usable by the classifier.
	Normalize Data	Connect the output from "Convert	This is important to scale numeric

6.		to Indicator Values" to a "Normalize Data" module. Set the **Transformation method to MinMax** and apply it to the **numeric feature columns** (Credit_Score, Income_(K), Loan_Amount), ensuring not to select the identifier or target label.	features (typically 0-1). It prevents features with larger dollar values (like Loan_Amount) from disproportionately influencing the model compared to features with smaller scales (like Credit_Score).
7.	**Split Data**	Connect the normalized data to a "Split Data" module. Set the "Fraction of rows in the first output dataset" to **0.7 (70%)**. This creates two outputs: 70% for training and 30% for testing.	As with regression, splitting data is essential. It provides a separate, **unseen dataset (30% testing set)** to evaluate the model's ability to generalize to new loan applications, rather than just overfitting to the training data.

Model Training & Evaluation

Step	What to Do	How to Do It	Why It's Important
1.	**Select Two-Class Algorithm (Untrained Model)**	Drag and drop a two-class classification algorithm module onto the canvas (e.g., "Two-Class Boosted Decision Tree"). This represents the untrained classifier.	For a **binary classification** task (predicting one of two outcomes, "Approved" or "Rejected"), a two-class algorithm is specifically designed to handle this type of problem.
2.	**Train Model**	Connect the selected two-class algorithm module to the **70% Training Dataset** output from the "Split Data" module. In the "Train Model" settings, specify the **Label column: Loan_Status**.	This step teaches the classification model. It uses the labeled training data to learn the patterns and relationships between loan application features and the Loan_Status, enabling it to classify future applications.
3.	**Score Model**	Connect the "Trained Model" output from the "Train Model" module to the "Score Model" module. Also, connect the **30% Testing Dataset** output from "Split Data" to the "Score Model" module.	This applies the trained classification model to the unseen test data. It generates "Scored Labels" (predicted approvals/rejections) for these new applications, simulating real-world performance.
4.	**Evaluate Model**	Connect the output from the "Score Model" module to an "Evaluate Model" module. This module will calculate performance metrics specific to classification.	This step is critical for assessing the classification model's accuracy. It provides key classification metrics (like Accuracy, Precision, Recall, F1-Score) to understand how well the model predicts loan approval.

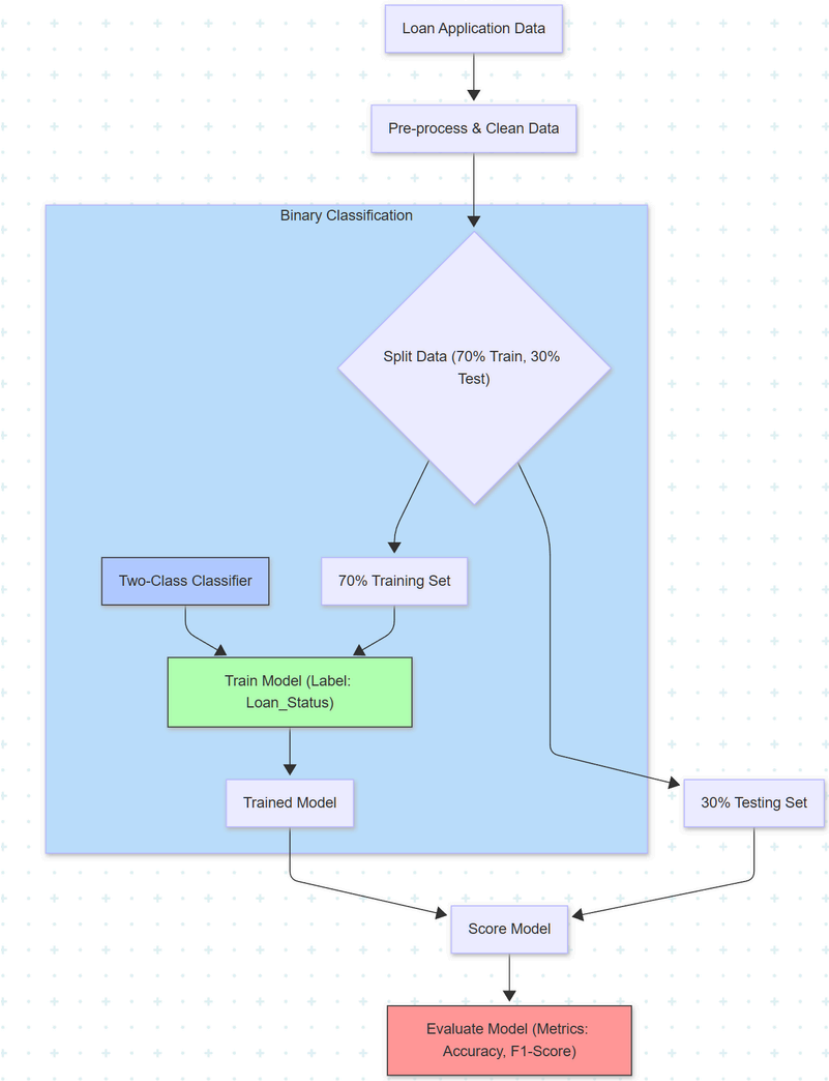
Azure ML Studio Implementation of the Home Loan Approval Classification Pipeline

This screenshot displays the actual classification pipeline constructed within Azure Machine Learning Studio. It provides a direct, practical view of how the detailed steps translate into a functional ML workflow in a cloud environment.

Screenshot 56: Flow Diagram: ML-Driven Autoscaling .

Conceptual Data Flow Diagram for the Classification Pipeline

This simplified diagram illustrates the high-level stages and logical data flow of the classification pipeline. Such diagrams are fundamental for visualizing system architecture, data movement, and component interactions.


Loan Application Data
Pre-process & Clean Data
Binary Classification
Split Data (70% Train, 30% Test)
Two-Class Classifier
70% Training Set
Train Model (Label: Loan_Status)
Trained Model
30% Testing Set
Score Model
Evaluate Model (Metrics: Accuracy, F1-Score)

Classification Evaluation Metrics

Unlike Regression, Classification models are evaluated using metrics that specifically focus on **correct** and **incorrect** categorical predictions. These metrics provide a nuanced view of the model's performance beyond simple accuracy.

Metric	Interpretation (Goal)
Accuracy	The general measure of model correctness, representing the **overall ratio** of correct predictions (both "Approved" and "Rejected") to total predictions.
Precision	Focuses on **minimizing false positives**. For loan approval, this means: *when the model predicts "Approved," how often is that prediction actually correct?* High Precision is critical to minimize financial risk by avoiding approving ineligible loans.
Recall	Focuses on **minimizing false negatives**. For loan approval, this means: *out of all the applications that were actually "Approved," how many did the model correctly identify?* High Recall is important to maximize business opportunity by not missing eligible loan applicants.
F1-Score	The harmonic mean (**balance**) **between Precision and Recall**. It is useful when an equal balance between minimizing false positives and false negatives is desired, especially with imbalanced datasets.

The Precision-Recall Trade-Off It is generally not possible to maximize both Precision and Recall simultaneously.

Tuning a model for **High Precision** (e.g., to minimize financial risk in loan approval) will likely result in **Lower Recall** (missing some eligible loan applicants).

Tuning for **High Recall** (e.g., to maximize business opportunity by approving more loans) will likely result in **Lower Precision** (approving more risky loans). The choice between prioritizing Precision or Recall depends entirely on the specific business objectives and the costs associated with each type of error.

Part A : Machine Learning Pipeline: Auto Price Training

3 Clustering Analysis

This table provides a step-by-step guide to building a K-Means Clustering model in Azure ML Studio to group flowers, designed for easy understanding and replication. Clustering is an Unsupervised Learning technique, meaning the model works with unlabeled data to discover inherent patterns.

Data Preparation

Step	What to Do	How to Do It	Why It's Important
1.	**Load Flower Data**	From the left-hand pane in Azure ML Studio, locate "**Sample data**" under "**Component** ". Choose the `flower dataset` (e.g., Flowers_Data.txt from an uploaded file, a sample, or web link). Drag the chosen dataset onto the pipeline canvas.	This is the raw input dataset containing flower characteristics (features) but **no predefined cluster labels** to predict, which is characteristic of unsupervised learning.

Data Pre-processing (Data Transformations)

Step	What to Do	How to Do It	Why It's Important
1.	**Edit Metadata (ID Column)**	Connect the dataset output to an "Edit Metadata" module. Select the `'Flower_ID'` column. Set **Fields: Clear Feature**.	The 'Flower_ID' is a unique identifier and should not be used as a feature for clustering. Clearing it prevents the algorithm from using a meaningless ID during distance calculations, which could skew results.
2.	**Normalize Data**	Connect the output from "Edit Metadata" to a "Normalize Data" module. Set the **Transformation method to MinMax** and apply it to the **feature numerical columns** (`Width, Height, Petal, Color_Code`).	This step is **crucial for clustering**. It ensures that all numerical features are on the same scale (e.g., 0 to 1). Without normalization, features with larger ranges (like petal length) would unfairly dominate the distance calculations, leading to inaccurate clusters.

Training

Step	What to Do	How to Do It	Why It's Important
1.	**Select K-Means Clustering Algorithm** (Untrained Model)	Drag and drop the "K-Means Clustering" module onto the canvas. In its settings, define the **number of clusters (K)**, for example, K=3 for three distinct flower types.	K-Means Clustering is a common **unsupervised learning** algorithm for grouping similar data points. Specifying K defines how many distinct groups the algorithm should attempt to find within the data.
2.	**Train Clustering Model**	Connect the "K-Means Clustering" module (Input 1 - Left Port) and the output from the "Normalize Data" module (Input 2 - Right Port) to a "Train Clustering Model" module. In the settings pane, ensure the **Label column dropdown is NOT SET (left completely blank)**.	This step teaches the clustering model to discover hidden structures. Since it's unsupervised, there's no "answer key" (label column). The algorithm learns patterns from the features (Width, Height, Petal, etc.) and creates clusters itself.

Evaluation & Post-Analysis

Step	What to Do	How to Do It	Why It's Important
1.	**Assign Data to Clusters**	Connect the "Trained Model" output from "Train Clustering Model" and the original normalized data to an "Assign Data to Clusters" module.	This module takes the trained K-Means model and applies it to each row of the input data, assigning each data point to its closest cluster center (e.g., Cluster 1, 2, or 3). This effectively labels the previously unlabeled data.
2.	**Evaluate Model**	Connect the output from the "Assign Data to Clusters" module to an "Evaluate Model" module. This module will calculate performance metrics for clustering.	This step assesses the quality of the formed clusters. Since there are no "correct" labels, evaluation focuses on internal cluster coherence using metrics like Average Distance to Cluster Center.

Clustering Evaluation Metric

For unsupervised learning like clustering, evaluation differs from supervised methods as there are **no predefined "correct" labels**. The focus is on the quality and compactness of the clusters themselves.

Metric	Interpretation (Goal)
Within-Cluster Sum of Squares (WCSS)	Measures the sum of the squared distances between each data point and the centroid (center) of its assigned cluster. **Goal: Minimize this value.** A lower WCSS indicates that data points within a cluster are very close to each other (high cohesion), signaling well-defined and tight groups.

Clustering Model Optimization
In practical applications, it is good practice to test a range of K values (the number of clusters). This iterative process, often guided by methods like the Elbow Method, is crucial for discovering the **optimal K value** that best fits the data's underlying structure, minimizing WCSS while maintaining simplicity.

Defining Categories from Cluster Output

After clustering, a key step is to interpret the results and assign meaningful names to the discovered groups.

1 Analyze Clustered Data Output

- **Action:** Export the final dataset from the "Assign Data to Clusters" module. This dataset will contain original features (Width, Height, Petal, Color_Code) along with a new column, typically named 'Assignments', containing the numerical Cluster ID (e.g., 1, 2, 3).
- **Purpose:** This data is ready for further analysis, usually in a data science notebook or spreadsheet, to understand the characteristics of each cluster.

2 Calculate Cluster Centroids (Means)

- **Action:** For each unique numerical Cluster ID, calculate the average (mean) value for every feature column (e.g., Average Width, Average Height, Average Petal, Average Color_Code).
- **Purpose:** These averages represent the centroid, or the "typical" profile, of a flower belonging to that specific cluster.

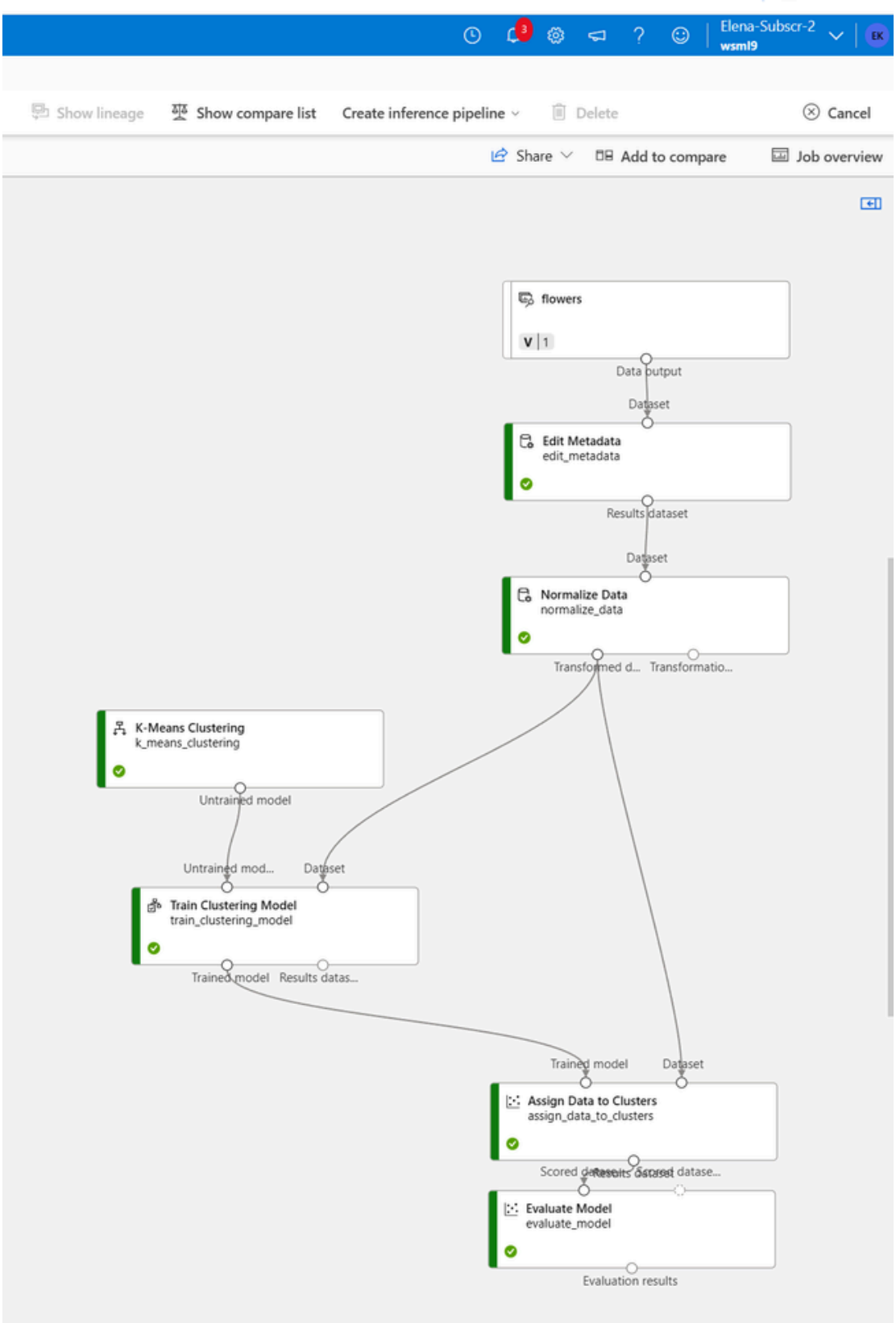
3 Assign Descriptive Names (The Category)

- **Action:** Compare the calculated mean values across all clusters to identify the defining characteristics of each group.
- **Result:** Translate these numerical profiles into descriptive, meaningful category names. For example:
  - **Cluster 1:** Lowest Height and Petal values -> "Small-Sized, Compact Flowers"
  - **Cluster 2:** Intermediate values for all metrics -> "Medium-Sized, Balanced Flowers"

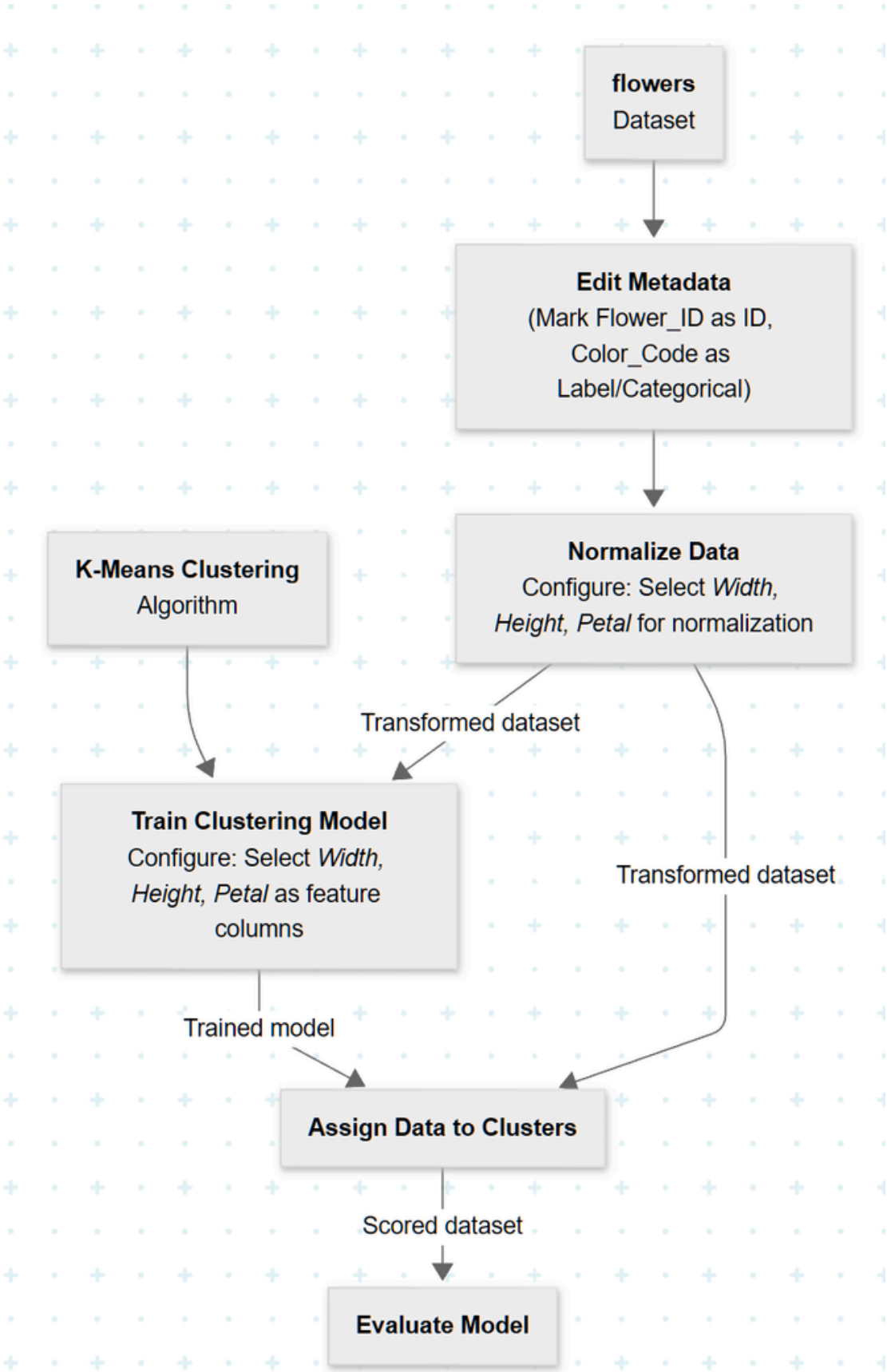
    - **Cluster 3:** Highest Height and Petal values -> "Large-Sized, Prominent Flowers"
- **Purpose:** This final step transforms raw machine learning output (cluster IDs) into valuable business insight (flower categories).

Azure ML Studio Implementation (Clustering Pipeline)

Screenshot 57: The actual clustering pipeline constructed within Azure ML Studio, demonstrating the practical workflow



Conceptual Data Flow Diagram (Clustering Pipeline)



Part A : Machine Learning Pipeline: Auto Price Training

4 Supervised and Unsupervised Learning

This section summarizes the key distinctions between different machine learning model types.

Type of Learning	Models Covered	Key Feature
Supervised Learning	Regression (Car Price), Classification (Loan Approval)	Uses **Labeled Data** for training. The goal is **Prediction**.
Unsupervised Learning	Clustering (Flower Grouping)	Uses **Unlabeled Data**. The goal is **Pattern Recognition** and **Discovery**.

Application: Use Cases

Learning Type	Relevant Use Case	Goal
Supervised	Predicting Server Failure	Predict future hardware failure for proactive replacement using historical, labeled logs.
Unsupervised	Customer Segmentation	Group users based on behavior to discover distinct market segments.

Impact of Unsupervised Learning on Training Unsupervised Learning (UL) significantly enhances supervised model training by:

- **Feature Engineering:** UL techniques (e.g., Dimensionality Reduction) simplify complex datasets, reducing training time and overfitting risk.
- **Anomaly Detection:** UL identifies outliers, which can be removed to improve model learning of normal patterns.
- **Generating Labels:** UL can provide initial labels for large unlabeled datasets, cost-effectively bootstrapping supervised model training.

Part B: Empowering Cloud Resource Allocation with ML

This part ties together all the principles demonstrated above, explaining how these models are used to drive better decisions in cloud resource allocation .

Machine Learning models empower effective cloud resource allocation by shifting management from reactive (responding to alerts) to proactive and predictive (forecasting future needs). This optimization is crucial for managing costs and maintaining high performance across multi-cloud environments (AWS/Azure).

ML Principle	Scenario Referenced	Cloud Resource Allocation Enhancement
Predictive Analytics (Regression)	Car Price Prediction	**Cost-Effective Autoscaling:** The model predicts **future compute demand** (e.g., number of EKS pods or Azure VMs) based on historical traffic patterns. This prediction is fed to the **Autoscaling Group**, allowing resources to be added or removed **just-in-time** instead of waiting for CPU thresholds to be crossed. This minimizes waste (over-provisioning) and latency (under-provisioning).
Classification Analysis	Home Loan Approval	**Service Reliability and Security:** The model is trained to **classify** incoming network traffic as "Normal" or "Attack/Anomaly." This allows services like **AWS WAF** or **Azure Firewall** to automatically block malicious traffic, protecting cloud resources and ensuring predictable performance for legitimate users.
Pattern Recognition (Clustering)	Flower Grouping	**Optimizing Storage Tiers:** The model can be applied to storage access logs (e.g., in **AWS S3** or **Azure Blob Storage**) to **group** data objects based on access frequency. Data in "Cluster A" (frequently accessed) stays in a high-cost, high-performance tier, while data in "Cluster B" (rarely accessed) is automatically moved to low-cost archival tiers, maximizing cost efficiency.
Model Evaluation	MAE, RMSE, F1-Score	**Risk Management:** Metrics (e.g., RMSE from Regression) quantify the model's prediction error. A low RMSE provides **confidence** in the prediction, making cloud decisions (like scaling down at night) a low-risk, automated process. High error indicates the need for human oversight before automating resource changes.

Summary of Impact

By leveraging these ML techniques, cloud decisions move from static provisioning (always allocating for peak load, wasting money) to dynamic, data-driven optimization, directly enhancing cost, performance, and scalability—the three pillars of cloud engineering.